

TECHNICAL PROJECT REPORT: OFFLINE-FIRST REST API CACHING APPLICATION

Project Title: REST API Data Fetching & Local Caching Application

Target Platform: Cross-Platform Mobile / Desktop (Flutter Framework)

Primary UI Paradigm: Material Design 3

Source Code Repository: <https://github.com/SudarshanDevadiga/API-Data-Fetcher-with-Local-Cache>

Remote Data Endpoint:

https://jsonplaceholder.typicode.com/posts

1. Executive Summary

This engineering report outlines the development and architectural strategy of a mobile application designed using the Flutter framework. The primary goal of the system is to deliver a resilient user experience when interacting with network-dependent data sources.

The system performs non-blocking asynchronous HTTP requests to retrieve public REST resources from JSONPlaceholder. To overcome intermittent connectivity or network outage, the application implements local persistence via SharedPreferences. When offline, the cached JSON payload is decoded from local disk storage and rendered dynamically using reactive state observers (FutureBuilder).

Primary Technical Deliverables

- **Asynchronous Network Integration:** Fetching REST payloads with non-blocking execution and time-out boundaries.
- **Local Persistence Layer:** Disk caching of raw JSON strings using SharedPreferences key-value store.
- **Multi-State UI Engine:** Reactive view updates reflecting live network data, loading skeletons, cached offline indicators, or full connection errors.
- **Modular Code Base:** Strict architectural separation between global application configuration and screen-level presentation.

2. Project Architecture & Directory Layout

The application isolates global application setup, routing, and styling from the core business logic, API requests, data models, and visual widgets.

Plaintext

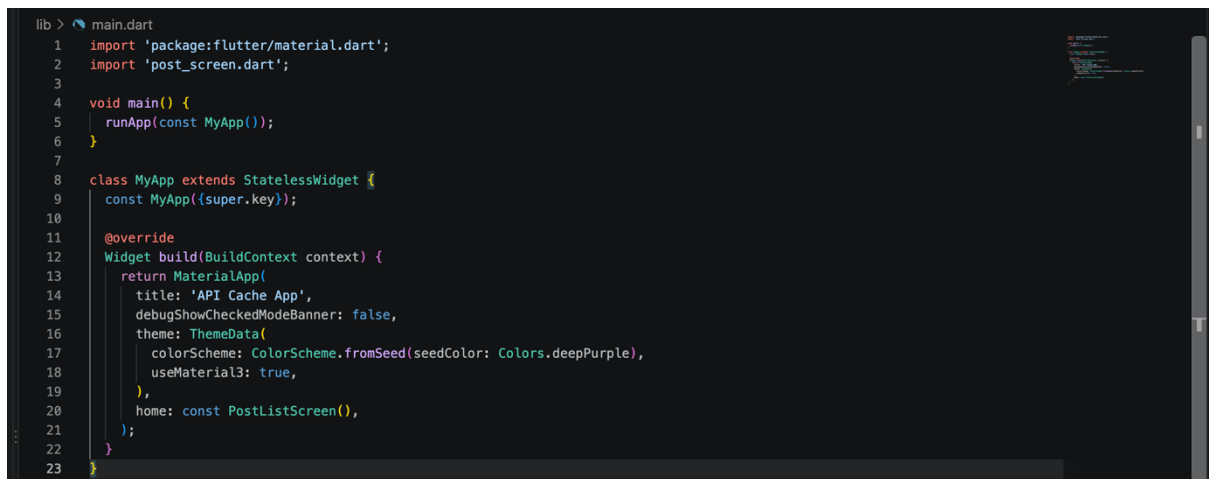
lib/

└─ main.dart # Entry point: App initialization, global Material 3 theme setup

└─ post_screen.dart # Implementation: Data models, API services, persistent cache, and screen widgets

3. Detailed Technical Implementation & Code Modules

3.1 Application Bootstrapping & Theme Definition (lib/main.dart)



```
lib > main.dart
1 import 'package:flutter/material.dart';
2 import 'post_screen.dart';
3
4 void main() {
5   runApp(const MyApp());
6 }
7
8 class MyApp extends StatelessWidget {
9   const MyApp({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return MaterialApp(
14       title: 'API Cache App',
15       debugShowCheckedModeBanner: false,
16       theme: ThemeData(
17         colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
18         useMaterial3: true,
19       ),
20       home: const PostListScreen(),
21     );
22   }
23 }
```

3.2 Data Deserialization Model (`lib/post\_screen.dart`)

```

lib > post_screen.dart
1 import 'dart:convert';
2 import 'package:flutter/material.dart';
3 import 'package:http/http.dart' as http;
4 import 'package:shared_preferences/shared_preferences.dart';
5
6 class Post {
7   final int id;
8   final String title;
9   final String body;
10
11   Post({
12     required this.id,
13     required this.title,
14     required this.body,
15   });
16
17   factory Post.fromJson(Map<String, dynamic> json) {
18     return Post(
19       id: json['id'] as int,
20       title: json['title'] as String,
21       body: json['body'] as String,
22     );
23   }
24
25   Map<String, dynamic> toJson() => {
26     'id': id,
27     'title': title,
28     'body': body,
29   };
30 }
31
32 class PostListScreen extends StatefulWidget {
33   const PostListScreen({super.key});
34
35   @override
36   State<PostListScreen> createState() => _PostListScreenState();
37 }
38
39 class _PostListScreenState extends State<PostListScreen> {
40   static const String _cacheKey = 'cached_posts';
41   late Future<List<Post>> _postsFuture;
42   bool _isFromCache = false;
43
44   @override
45   void initState() {
46     super.initState();
47     _postsFuture = fetchAndCachePosts();
48   }
49
50   void _refreshPosts() {
51     setState(() {
52       _postsFuture = fetchAndCachePosts();
53     });
54   }
55
56   Future<List<Post>> fetchAndCachePosts() async {
57     const String url = 'https://jsonplaceholder.typicode.com/posts';
58     final prefs = await SharedPreferences.getInstance();
59
60     try {
61       final response = await http.get(Uri.parse(url)).timeout(
62         const Duration(seconds: 8),
63       );
64
65       if (response.statusCode == 200) {
66         await prefs.setString(_cacheKey, response.body);
67         _isFromCache = false;
68
69         final List<dynamic> jsonList = jsonDecode(response.body);
70         return jsonList.map((json) => Post.fromJson(json)).toList();
71       } else {
72         throw Exception('Server error: ${response.statusCode}');
73       }
74     } catch (e) {
75       final String? cachedJson = prefs.getString(_cacheKey);
76
77       if (cachedJson != null && cachedJson.isNotEmpty) {
78         _isFromCache = true;
79         final List<dynamic> jsonList = jsonDecode(cachedJson);
80         return jsonList.map((json) => Post.fromJson(json)).toList();
81       }
82
83       rethrow;
84     }
85   }
86 }

```

```

87 @override
88 Widget build(BuildContext context) {
89   final theme = Theme.of(context);
90
91   return Scaffold(
92     backgroundColor: theme.colorScheme.surfaceContainerLowest,
93     appBar: AppBar(
94       centerTitle: false,
95       scrolledUnderElevation: 2.0,
96       title: Column(
97         crossAxisAlignment: CrossAxisAlignment.start,
98         children: [
99           Text(
100             'Explore Feeds',
101             style: theme.textTheme.titleLarge?.copyWith(
102               fontWeight: FontWeight.bold,
103             ),
104           ),
105           Text(
106             'JSONPlaceholder • Cached with LocalStorage',
107             style: theme.textTheme.bodySmall?.copyWith(
108               color: theme.colorScheme.onSurfaceVariant,
109             ),
110           ),
111         ],
112       ),
113       actions: [
114         IconButton.filledTonal(
115           icon: const Icon(Icons.refresh_rounded, size: 20),
116           onPressed: _refreshPosts,
117           tooltip: 'Refresh Feed',
118         ),
119         const SizedBox(width: 12),
120       ],
121     ),
122     body: RefreshIndicator(
123       onRefresh: () async => _refreshPosts(),
124       child: FutureBuilder<List<Post>>(
125         future: _postsFuture,
126         builder: (context, snapshot) {
127           // 1. Loading State
128           if (snapshot.connectionState == ConnectionState.waiting) {
129             return ListView.builder(
130               itemCount: 6,
131               itemBuilder: (_, __) => const _SkeletonCard(),
132             );
133           }
134
135           // 2. Error State (No network & No cache)
136           if (snapshot.hasError) {
137             return Center(
138               child: Padding(
139                 padding: const EdgeInsets.all(24.0),
140                 child: Column(
141                   mainAxisAlignment: MainAxisAlignment.center,
142                   children: [
143                     Container(
144                       padding: const EdgeInsets.all(20),
145                       decoration: BoxDecoration(
146                         color: theme.colorScheme.errorContainer.withOpacity(0.4),
147                         shape: BoxShape.circle,
148                       ),
149                       child: Icon(
150                         Icons.wifi_off_rounded,
151                         size: 40,
152                         color: theme.colorScheme.error,
153                       ),
154                     ),
155                     const SizedBox(height: 20),
156                     Text(
157                       'Connection Unavailable',
158                       style: theme.textTheme.titleMedium?.copyWith(
159                         fontWeight: FontWeight.bold,
160                       ),
161                     ),
162                     const SizedBox(height: 8),
163                     Text(
164                       'Unable to connect to the network and no local cache was found.',
165                       textAlign: TextAlign.center,
166                       style: theme.textTheme.bodyMedium?.copyWith(
167                         color: theme.colorScheme.onSurfaceVariant,
168                       ),
169                     ),
170                   ],
171                 ),
172             );
173           }
174
175           // Success State (Not shown in the provided code)
176           // return ListView.builder(
177           //   itemCount: _postsFuture.data?.length,
178           //   itemBuilder: (context, index) => PostCard(
179           //     post: _postsFuture.data?[index],
180           //   ),
181           // );
182         },
183       ),
184     ),
185   );
186 }

```

```

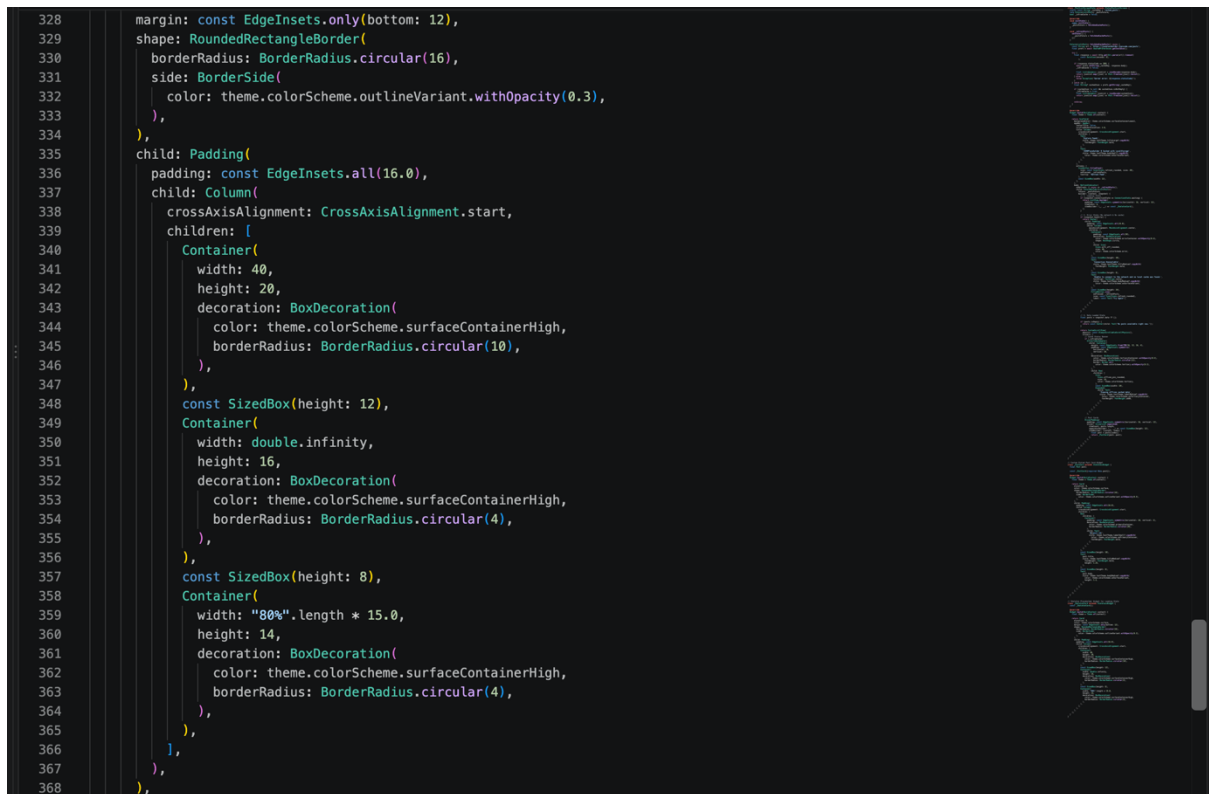
170         ),
171         const SizedBox(height: 24),
172         FilledButton.icon(
173           onPressed: _refreshPosts,
174           icon: const Icon(Icons.refresh_rounded),
175           label: const Text('Try Again'),
176         ),
177       ],
178     ),
179   );
180 };
181
182
183 // 3. Data Loaded State
184 final posts = snapshot.data ?? [];
185
186 if (posts.isEmpty) {
187   return const Center(child: Text('No posts available right now.'));
188 }
189
190 return CustomScrollView(
191   physics: const AlwaysScrollableScrollPhysics(),
192   slivers: [
193     // Cache Status Banner
194     if (_isFromCache)
195       SliverToBoxAdapter(
196         child: Container(
197           margin: const EdgeInsets.fromLTRB(16, 12, 16, 4),
198           padding: const EdgeInsets.symmetric(
199             horizontal: 16,
200             vertical: 10,
201           ),
202           decoration: BoxDecoration(
203             color: theme.colorScheme.tertiaryContainer.withOpacity(0.5),
204             borderRadius: BorderRadius.circular(12),
205             border: Border.all(
206               color: theme.colorScheme.tertiary.withOpacity(0.3),
207             ),
208           ),
209           child: Row(
210             children: [
211               Icon(
212                 Icons.offline_pin_rounded,
213                 size: 18,
214                 color: theme.colorScheme.tertiary,
215               ),
216               const SizedBox(width: 10),
217               Expanded(
218                 child: Text(
219                   'Viewing offline cached data',
220                   style: theme.textTheme.labelMedium?.copyWith(
221                     color: theme.colorScheme.onTertiaryContainer,
222                     fontWeight: FontWeight.w600,
223                   ),
224                 ),
225               ),
226             ],
227           ),
228         ),
229       ),
230
231     // Post Cards
232     SliverPadding(
233       padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
234       sliver: SliverList.separated(
235         itemCount: posts.length,
236         separatorBuilder: (_, __) => const SizedBox(height: 12),
237         itemBuilder: (context, index) {
238           final post = posts[index];
239           return _PostCard(post: post);
240         },
241       ),
242     ),
243   ],
244 );
245
246 ),
247

```

```

248     );
249   }
250 }
251
252 // Custom Styled Post Card Widget
253 class _PostCard extends StatelessWidget {
254   final Post post;
255
256   const _PostCard({required this.post});
257
258   @override
259   Widget build(BuildContext context) {
260     final theme = Theme.of(context);
261
262     return Card(
263       elevation: 0,
264       color: theme.colorScheme.surface,
265       shape: RoundedRectangleBorder(
266         borderRadius: BorderRadius.circular(16),
267         side: BorderSide(
268           color: theme.colorScheme.outlineVariant.withOpacity(0.5),
269         ),
270       ),
271       child: Padding(
272         padding: const EdgeInsets.all(16.0),
273         child: Column(
274           crossAxisAlignment: CrossAxisAlignment.start,
275           children: [
276             Row(
277               children: [
278                 Container(
279                   padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
280                   decoration: BoxDecoration(
281                     color: theme.colorScheme.primaryContainer,
282                     borderRadius: BorderRadius.circular(20),
283                   ),
284                   child: Text(
285                     '${post.id}',
286                     style: theme.textTheme.labelSmall?.copyWith(
287                       color: theme.colorScheme.onPrimaryContainer,
288                       fontWeight: FontWeight.bold,
289                     ),
290                   ),
291                 ),
292               ],
293             ),
294             const SizedBox(height: 10),
295             Text(
296               post.title,
297               style: theme.textTheme.titleMedium?.copyWith(
298                 fontWeight: FontWeight.bold,
299                 height: 1.25,
300               ),
301             ),
302             const SizedBox(height: 8),
303             Text(
304               post.body,
305               style: theme.textTheme.bodyMedium?.copyWith(
306                 color: theme.colorScheme.onSurfaceVariant,
307                 height: 1.4,
308               ),
309             ),
310           ],
311         ),
312       ),
313     );
314   }
315 }
316
317 // Skeleton Placeholder Widget for Loading State
318 class _SkeletonCard extends StatelessWidget {
319   const _SkeletonCard();
320
321   @override
322   Widget build(BuildContext context) {
323     final theme = Theme.of(context);
324
325     return Card(
326       elevation: 0,
327       color: theme.colorScheme.surface,

```



3.3 Asynchronous Fetching & Fallback Caching Logic

The core logic resides inside `fetchAndCachePosts()` [cite: 1]. The execution sequence operates as follows:

1. Remote HTTP Execution: The client initiates an asynchronous `http.get` request targeting the REST API with an 8-second execution timeout.
2. Cache Write on Success: Upon receiving an HTTP status code `200`, the raw JSON response payload is written to `SharedPreferences` under the key `cached_posts`, and the `_isFromCache` flag is set to `false`.
3. Exception Catch & Cache Read: If a socket exception, network timeout, or server connection failure occurs, the catch block intercepts the exception and checks `SharedPreferences` for `cached_posts`.
4. Offline Processing: If cached data exists, it deserializes the saved string into `Post` objects and sets `_isFromCache` to `true`.
5. Unhandled Error: If no cached payload exists on disk and the network fails, the error is rethrown to display an offline error screen.

4. Component Responsibility Matrix

Component / Class	Type	Primary Architectural Function
---	---	---

| ``FutureBuilder<List<Post>>`` | Reactive Widget | Listens to ``_postsFuture`` and dynamically builds UI for ``waiting``, ``hasError``, or ``hasData`` states. |

| ``SharedPreferences`` | Data Storage | Provides disk-based key-value persistence for stringified JSON payloads across application reboots. |

| ``http.get`` | Network Client | Handles HTTP communication with timeout boundaries. |

| ``RefreshIndicator`` | UI Interaction | Captures user swipe-down gestures to re-trigger the API fetch process. |

| ``CustomScrollView`` & ``Slivers`` | Layout Engine | Renders scrolling lists and injects contextual header banners (e.g., offline mode warnings). |

5. System UI Behavior across Operational States

State A: Active Internet Connection (Online Mode)

- * Fetches live data from the JSONPlaceholder REST endpoint.
- * Maps response items into structured Material 3 cards showing post IDs, titles, and body content.
- * Updates local disk storage automatically in the background.

State B: Disconnected with Saved Cache (Offline Mode)

- * Retains access to post feeds even without active network interface.
- * Reads previously persisted JSON from ``SharedPreferences``.
- * Displays a contextual tertiary banner: ``"Viewing offline cached data"``.

State C: Disconnected without Cache (Uncached Error Mode)

- * Triggers when the application is opened offline with an empty cache.
- * Replaces the post feed with a dedicated connection error interface.
- * Provides a ``"Try Again"`` interactive button to retry network calls.

6. Technical Issues & Applied Solutions

Observed Issue	Root Cause	Implemented Resolution
Missing Package Dependencies	<code>`http`</code> and <code>`shared_preferences`</code> libraries were missing from <code>`pubspec.yaml`</code> [cite: 1].	Added both dependencies to <code>`pubspec.yaml`</code> and executed <code>`flutter pub get`</code> [cite: 1].
Static Analysis Error	Target file lacked the mandatory <code>`.dart`</code> file extension [cite: 1].	Renamed the file to <code>`post_screen.dart`</code> to re-enable Dart language analysis.

| ****Infinite API Call Loops**** | Invoking `fetchAndCachePosts()` directly within `build()` forced re-fetching on every frame render. | Assigned `_postsFuture` inside `initState()` so network execution triggers once during initial widget creation. |

7. Key Engineering Takeaways

- * **Offline-First Architecture:** Storing API payloads locally ensures the software remains operational in network-constrained environments.
- * **Widget Lifecycle Discipline:** Binding future execution to `initState()` prevents excessive API calls and UI render loops.
- * **User Experience Clarity:** Presenting dynamic loading skeletons and clear offline indicators maintains visual feedback during connection changes.

